Banking Data Pipeline and Dashboard

From raw transactions to numbers a bank can trust

An automated ELT pipeline built with MySQL, Python and Power BI on the Berka bank dataset

Prakhar Gupta

Data Analytics Portfolio Project

Contents

1 The project at a glance **4**

1.1 What this project is, in plain words . . . 4

1.2 Key numbers . . . 4

1.3 The path of the data, in one picture . . . 5

2 The problem and the objective **5**

2.1 The problem . . . 5

2.2 The objective . . . 5

2.3 Why this matters to a bank . . . 6

3 Tools used **6**

4 The data: what we started with **6**

4.1 About the dataset . . . 6

4.2 Table 1: `client`, the customers . . . 7

4.3 Table 2: `account`, the bank accounts . . . 7

4.4 Table 3: `disp`, who is linked to which account . . . 7

4.5 Table 4: `district`, facts about each region . . . 7

4.6 Table 5: `loan`, loans given to accounts . . . 8

4.7 Table 6: `trans`, every transaction . . . 8

4.8 How the tables connect . . . 9

4.9 The quirks of the data . . . 9

5 The design: how the project is organised **10**

5.1 Two databases . . . 10

5.2 ETL or ELT? . . . 10

5.3 The seven steps . . . 10

6 Step 1: Extract and load **10**

6.1 Goal . . . 10

6.2 Creating the two databases and the load log . . . 11

6.3 Loading with Python . . . 11

6.4 Safe connection: no password in the code . . . 12

6.5 Indexes . . . 12

6.6 Result of Step 1 . . . 12

7 Step 2: Transform with SQL views **13**

7.1 What is a view? . . . 13
7.2 Problem 1: dates stored as numbers . . . 13
7.3 Problem 2: gender and birth date hidden in one number . . . 14
7.4 Problem 3: age . . . 14
7.5 Problem 4: Czech codes . . . 15
7.6 The views we created . . . 15
7.7 Two clever pieces of SQL . . . 15

8 Step 3: Stored procedures and reporting tables 16
8.1 The problem with views alone . . . 16
8.2 What is a stored procedure? . . . 17
8.3 The four procedures . . . 17
8.4 Building the reporting tables . . . 17
8.5 Why tables and not just views? . . . 18
8.6 The master procedure . . . 18
8.7 The helper that records a check . . . 18
8.8 Two helper tables . . . 19

9 Step 4: Data quality checks 19
9.1 Goal . . . 19
9.2 How a check works . . . 19
9.3 The 14 checks . . . 20
9.4 The five groups explained simply . . . 20
9.5 Why the thresholds differ . . . 20
9.6 Results in a chart . . . 21

10 Step 5: Root cause analysis 21
10.1 Why it is needed . . . 21
10.2 The method: three tests . . . 21
10.3 Finding 1: blank bank and blank counterparty account . . . 22
10.4 Finding 2: blank operation . . . 22
10.5 Finding 3: blank k_symbol . . . 23
10.6 Finding 4: negative balances (a passed check, reviewed anyway) . . . 23
10.7 Other discoveries . . . 24
10.8 Recommendations . . . 24
10.9 How certain are these conclusions? . . . 24

11 Step 6: Automation 24

11.1 Goal . . . 24
11.2 What runs, in order . . . 25
11.3 First-time setup versus every later run . . . 25
11.4 Logging at every level . . . 25
11.5 Scheduling . . . 26

12 Step 7: The Power BI dashboard 26
12.1 Goal . . . 26
12.2 Connecting to the data . . . 26
12.3 The data model (relationships) . . . 27
12.4 The 12 measures . . . 27
12.5 Page 1: Overview . . . 28
12.6 Page 2: Customers and Loans . . . 29
12.7 Page 3: Data Quality . . . 29

13 What we created and what we kept 30
13.1 Everything in the project, in order of creation . . . 30
13.2 What each layer is for . . . 30
13.3 Why each layer is kept . . . 31
13.4 Why `refresh_log` and `run_log` are different . . . 31
13.5 Views and tables in Power BI . . . 31

14 Results, limitations and lessons 31
14.1 What was achieved . . . 31
14.2 Limitations (stated honestly) . . . 31
14.3 Lessons learned . . . 32

A Glossary 32

B Full code listings 33
B.1 `sql/00_create_databases.sql` . . . 33
B.2 `sql/01_setup.sql` . . . 33
B.3 `sql/02_root_cause.sql` . . . 36
B.4 `sql/03_data_dictionary.sql` . . . 37
B.5 `python/run_pipeline.py` . . . 37
B.6 `run_pipeline.bat` . . . 39

1 The project at a glance

1.1 What this project is, in plain words

A bank collects a huge amount of data every day: who its customers are, which accounts they have, every deposit and withdrawal, and every loan. This data arrives as plain files that are messy, full of codes, and not ready to be read by a manager.

This project builds a small **factory** that takes those messy files and turns them into clean, checked, easy-to-read answers. The factory runs from a single command. At the end, a Power BI dashboard shows the answers on screen, and a **data quality page** shows how much the data can be trusted.

Think of it like this

Imagine a restaurant. Vegetables arrive from the farm dirty and in boxes. The kitchen washes them, cuts them following a recipe, cooks them, and puts the finished meals on a counter. A food inspector checks that every dish is safe. Finally, a menu board tells customers what is available. This project does the same with bank data:

Restaurant	This project
Boxes of dirty vegetables	Raw CSV files
Storage room	The `staging` database
Recipes	SQL *views*
Cook who follows the recipes	*Stored procedures*
Meals on the counter	Clean `reporting` tables
Food inspector	14 data quality checks
Menu board	Power BI dashboard

1.2 Key numbers

Item	Value
Bank clients	5,369
Bank accounts	4,500
Transactions processed	1,056,320
Loans	682
Districts (regions of the country)	77
Source files loaded	6
SQL views created	6
Stored procedures created	4
Automated data quality checks	14 (10 pass, 4 fail)
Power BI measures (formulas)	12
Dashboard pages	3

1.3 The path of the data, in one picture

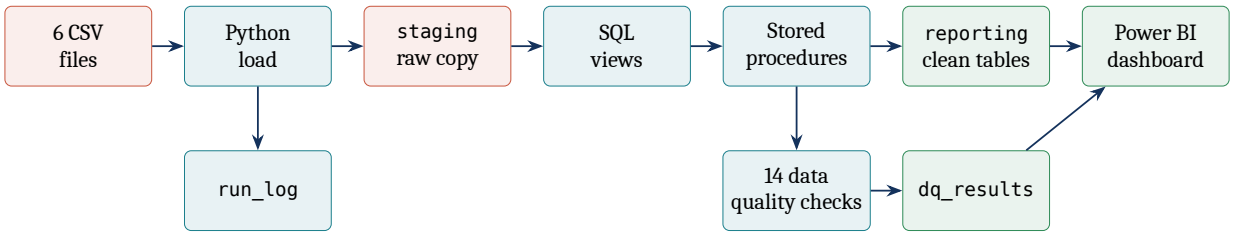


Figure 1: The whole project on one page. Red is raw data, green is clean data, teal is something that does work.

2 The problem and the objective

2.1 The problem

Raw bank data is hard to use for four reasons:

1. **It is written in codes.** For example, a transaction type is stored as `PRIJEM` or `VYDAJ`, which are Czech words for “credit” and “withdrawal”.
2. **Information is hidden inside numbers.** A date is stored as the number `930105` and a client’s gender is hidden inside the birth number.
3. **It is huge.** One table has over one million rows, which is slow to use directly in a dashboard.
4. **Nobody knows if it is correct.** Are there duplicate transactions? Transactions that belong to no account? Missing values? Without checking, a wrong number can reach a decision maker.

2.2 The objective

In one line

Build an automated, repeatable reporting pipeline for a retail bank that loads raw files, cleans and checks them with SQL, explains every data problem it finds, and serves trusted numbers to a Power BI dashboard.

The project is therefore built around five goals:

1. **Load** raw files safely and keep a record of every load.
2. **Transform** codes and encoded values into readable business fields, keeping the raw data untouched.
3. **Prepare** fast, clean reporting tables for the dashboard.
4. **Check** data quality automatically, and find the root cause of every failure.
5. **Present** the results in a dashboard, including the data quality status.

2.3 Why this matters to a bank

Why a bank wants this

Banks make decisions about money and about people. They must show regulators where each number came from and that it is complete and accurate. A pipeline that is repeatable, logged and checked is a small version of the discipline that banking data standards (such as the Basel Committee's BCBS 239 principles on risk data aggregation) ask for. It also saves the time analysts would otherwise spend copying data between spreadsheets by hand.

3 Tools used

Tool	Used for	Where in the project
MySQL 8	The database that stores all data	Databases `staging` and `reporting`
MySQL Workbench	Writing and running SQL	Running the setup script and the checks
SQL (views, stored procedures)	Cleaning, rules and automation inside the database	Steps 2, 3 and 4
Python 3	Reading files and running the pipeline	`run_pipeline.py`
pandas	Reading large CSV files in chunks	Step 1
SQLAlchemy and PyMySQL	Python's connection to MySQL	Step 1 and Step 3
Power BI Desktop	Building the dashboard	Step 7
DAX	Formulas for dashboard numbers	12 measures
Windows Command Prompt	Running the pipeline	Step 6
Windows Task Scheduler (optional)	Running the pipeline on a schedule	Step 6

All tools are free. Power BI Desktop is free, and automatic scheduled refresh would need the paid Power BI Service.

4 The data: what we started with

4.1 About the dataset

The project uses the **Berka dataset**, a public dataset released for a data discovery challenge by a Czech bank. It describes about 5,400 clients, 4,500 accounts, over a million transactions and 682 loans between 1993 and 1998. The dataset is old and anonymised, which makes it safe to use for learning, but it behaves like real banking data: coded values, missing values and quirks.

The data came as six CSV files, separated by semicolons. Each file is one table.

4.2 Table 1: `client`, the customers

Column	Example	Meaning
`client_id`	1	Unique number of the customer
`birth_number`	706213	Birth date and gender *encoded*: YYMMDD, with 50 added to the month for women
`district_id`	18	Region where the client lives

Rows: 5,369.

4.3 Table 2: `account`, the bank accounts

Column	Example	Meaning
`account_id`	1	Unique number of the account
`district_id`	18	Region of the branch where the account was opened
`frequency`	POPLATEK MESICNE	How often statements are issued (monthly, weekly, or after each transaction)
`date`	930101	Account opening date as a number, YYMMDD

Rows: 4,500.

4.4 Table 3: `disp`, who is linked to which account

A client can be the **owner** of an account, or a **disponent**, which is an authorised user.

Column	Example	Meaning
`disp_id`	1	Unique number of the link
`client_id`	1	The client
`account_id`	1	The account
`type`	OWNER	OWNER or DISPONENT

Rows: 5,369. There are more clients than accounts (5,369 against 4,500) because some accounts have an extra authorised user.

4.5 Table 4: `district`, facts about each region

This table has 77 rows and 16 columns, named `A1` to `A16`, which is not helpful. We rename them in our project:

Raw name	Our name	Meaning
A1	district_id	Region number
A2	district_name	Region name
A3	region	Larger area, e.g. Prague, north Bohemia
A4	population	Number of inhabitants
A9	num_cities	Number of cities
A10	urban_ratio_pct	Share of urban population
A11	avg_salary	Average salary
A12, A13	unemployment_rate_1995, ..._1996	Unemployment rate
A14	entrepreneurs_per_1000	Entrepreneurs per 1,000 people
A15, A16	crimes_1995, crimes_1996	Number of crimes

Columns A5 to A8 (counts of municipalities by size) are not needed for our analysis and are left out of the clean table.

4.6 Table 5: loan, loans given to accounts

Column	Example	Meaning
loan_id	5314	Unique number of the loan
account_id	1787	The account that took the loan
date	930705	Date the loan was granted, YYMMDD
amount	96396	Loan amount
duration	12	Length of the loan in months
payments	8033	Monthly payment
status	B	A, B, C or D (see below)

Rows: 682. The four statuses are: **A** finished and paid, **B** finished and not paid, **C** running and fine, **D** running and in debt.

4.7 Table 6: trans, every transaction

This is the biggest and most important table, with **1,056,320 rows**.

Column	Example	Meaning
`trans_id`	695247	Unique number of the transaction
`account_id`	2378	The account
`date`	930101	Date, YYMMDD
`type`	PRIJEM	PRIJEM = credit (money in), VYDAJ = withdrawal (money out), VYBER = cash withdrawal
`operation`	VKLAD	Kind of operation, e.g. cash deposit, cash withdrawal, card withdrawal, transfer
`amount`	700	Money moved
`balance`	700	Account balance after the transaction
`k_symbol`	UROK	Purpose code, e.g. interest, pension, loan payment, insurance (often blank)
`bank`	AB	Other bank involved (transfers only)
`account`	41403269	Other account involved (transfers only)

The examples above are illustrative values showing the format, not specific rows.

4.8 How the tables connect

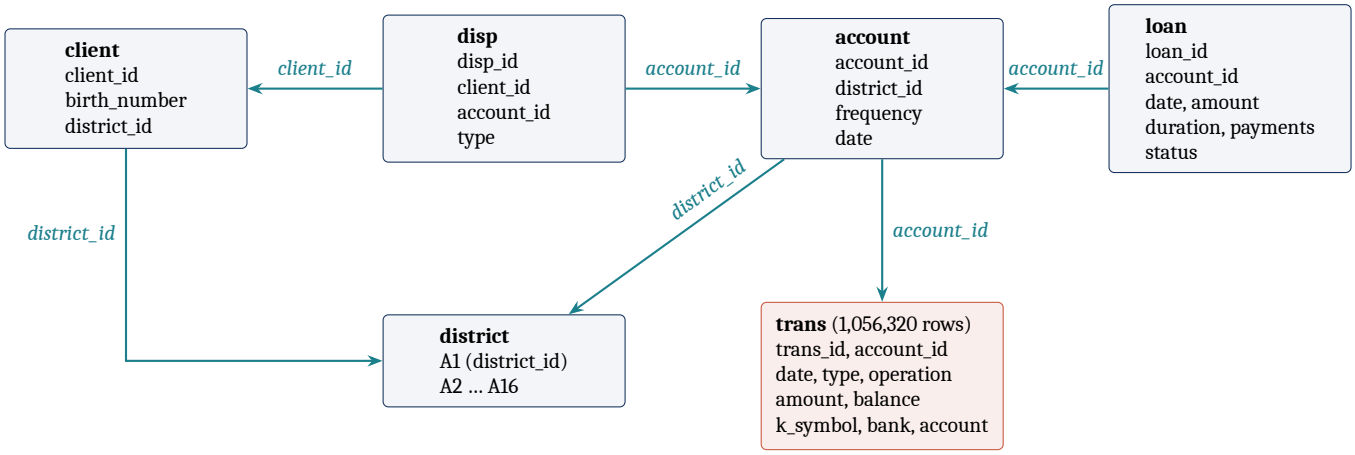


Figure 2: The six source tables. An arrow means "points to": for example every row in `disp` says which client and which account it links.

4.9 The quirks of the data

These are the reasons the transformation step exists:

- **Dates are numbers.** `930105` means 5 January 1993.
- **Gender and birth date are hidden** inside `birth_number`.
- **Czech codes** such as PRIJEM, VYDAJ, VYBER and the loan letters A to D.
- **Cryptic column names** in `district`.
- **Blank values** in `bank`, `account`, `operation` and `k_symbol`. Step 5 shows that these are not accidents.

- **One extremely large table** (`trans`).

5 The design: how the project is organised

5.1 Two databases

We create two databases inside MySQL, because raw and clean data must never be mixed.

Database	Contains	Rule
`staging`	Raw copies of the six files, plus the load log	Never edited after loading
`reporting`	Views, clean tables, logs and quality results	Everything here can be rebuilt at any time from `staging`

5.2 ETL or ELT?

ETL means Extract, Transform, Load. ELT changes the order: Extract, Load, then Transform *inside the database*. This project is ELT:

1. **Extract and Load:** Python copies the files into `staging`, unchanged.
2. **Transform:** SQL views and stored procedures clean the data inside MySQL.

Loading first has a big advantage: the original data stays safe. If a cleaning rule turns out to be wrong, we fix the rule and rebuild, with no need to read the files again.

5.3 The seven steps

Step	Name	What happens
1	Extract and load	Python loads six CSV files into `staging` and logs each load
2	Transform	SQL views decode dates, gender, age and codes
3	Build reporting	Stored procedures save the views as fast, clean tables
4	Data quality	14 automated checks, with results stored
5	Root cause analysis	Investigate why four checks failed
6	Automate	One command runs everything, with logging
7	Dashboard	Power BI reads the clean tables

6 Step 1: Extract and load

6.1 Goal

Copy the data from the files into the database exactly as it is. We do not clean anything yet. The goal is a faithful raw copy and a record that proves the copy is complete.

6.2 Creating the two databases and the load log

The script `00_create_databases.sql` runs once. It creates the databases and a table that records every load.

Main code

```
CREATE DATABASE IF NOT EXISTS staging;                    (1)
CREATE DATABASE IF NOT EXISTS reporting;                  (1)

CREATE TABLE IF NOT EXISTS staging.run_log (              (2)
    run_id        INT AUTO_INCREMENT PRIMARY KEY,         (3)
    run_time      DATETIME DEFAULT CURRENT_TIMESTAMP,     (4)
    table_name    VARCHAR(50),                            (5)
    rows_loaded   INT,                                    (5)
    status        VARCHAR(10),                            (6)
    error_message TEXT                                    (6)
);
```

What this code does

(1) Creates the two empty databases. `IF NOT EXISTS` means running it twice does no harm.
(2) Creates the logbook table `run_log` inside `staging`.
(3) Every log line gets its own number automatically.
(4) The time of the load is filled in by the database itself.
(5) Which file was loaded, and how many rows went in.
(6) Whether it worked (`SUCCESS` or `FAIL`) and the error text if it did not.

6.3 Loading with Python

The Python script `run_pipeline.py` does the loading. The listing below is shortened for reading; the complete script is in Appendix B.

Main code

```
def load_staging():
    for t in TABLES:                                      (1)
        total = 0
        for i, chunk in enumerate(pd.read_csv(            (2)
                find_file(t), sep=";", chunksize=100_000)):
            chunk.to_sql(t, engine, index=False,          (3)
                if_exists="replace" if i == 0 else "append")
            total += len(chunk)                           (4)
        write_run_log(t, total, "SUCCESS")                (5)
```

What this code does

(1) Repeat for each of the six tables: account, client, disp, district, loan, trans.
(2) Open the CSV file (values separated by `;`) and read it in **chunks of 100,000 rows**. The `trans` file has over a million rows, and reading it in pieces keeps the computer's memory small.
(3) Write the chunk into MySQL. The *first* chunk replaces any old table (a fresh start), and every later chunk is *appended*. This is why running twice never duplicates data.
(4) Keep a running count of the rows loaded.
(5) Write one line in the logbook: table name, row count, SUCCESS. In the full script, an error writes FAIL with the error text and stops the run.

6.4 Safe connection: no password in the code

The database password is never written in the code, so it can never leak to GitHub.

Main code

```
password = os.environ.get("MYSQL_PASSWORD")                    (1)
if not password:                                               (2)
    sys.exit("Set the MYSQL_PASSWORD variable first.")

engine = create_engine(URL.create(                             (3)
    "mysql+pymysql", username="root", password=password,
    host="localhost", port=3306, database="staging"))          (4)
```

What this code does

(1) Reads the password from a hidden setting on the computer (an *environment variable*) instead of from the code.

(2) If the setting is missing, the program stops at once with a clear message.

(3) Builds the connection with SQLAlchemy's `URL.create`. It handles special characters such as `@` in a password correctly, which a plain text address does not.

(4) Connects to the `staging` database on this computer.

6.5 Indexes

After loading, the script adds indexes, for example on `trans(account_id)` and `trans(trans_id)`. An index works like the index at the back of a book: instead of reading a million rows to find one account, MySQL jumps straight to the right place.

Main code

```
for tbl, col in INDEXES:                                       (1)
    conn.execute(text(f"CREATE INDEX idx_{tbl}_{col} "         (2)
                      f"ON staging.{tbl} ({col})"))
```

What this code does

(1) Goes through a list of (table, column) pairs, such as (`trans`, `account_id`).

(2) Builds one index for each pair. If an index already exists, the full script writes a warning to the log and moves on instead of crashing.

6.6 Result of Step 1

After a clean run, the row counts in `staging` matched the files exactly:

Staging table	Rows
`account`	4,500
`client`	5,369
`disp`	5,369
`district`	77
`loan`	682
`trans`	1,056,320

Why a bank wants this

- **Audit trail:** the bank can show what was loaded, when, and how many rows.
- **Failures are visible:** during testing one run loaded only 956,320 of 1,056,320 transaction rows because of an interrupted run. The row count and the log exposed it. Without this, reports would have been silently wrong.
- **No manual copying:** fewer human mistakes.

7 Step 2: Transform with SQL views

7.1 What is a view?

A **view** is a saved SQL query with a name. It looks like a table, and you can query it like a table, but it does not store data. Each time you use it, MySQL runs the saved query on the underlying tables.

Think of it like this

A view is a recipe. The vegetables (the staging tables) stay as they are. The recipe says how to wash, cut and combine them. Everybody who follows the same recipe gets the same dish.

Why views?

- The cleaning logic is written once and reused everywhere.
- Every report uses the same definition of, say, "age", so numbers agree.
- Raw data is untouched. If a rule is wrong, change the view.
- A reviewer can open the view and see exactly how a number was made.

The views live in the `reporting` database. The script `01_setup.sql` creates them.

7.2 Problem 1: dates stored as numbers

The number `930105` means 5 January 1993. We turn it into a real date.

Main code

```
STR_TO_DATE(                                          (3)
    CONCAT('19',                                      (2)
           LPAD(date, 6, '0')),                       (1)
    '%Y%m%d')                                         (3)
```

What this code does

(1) `LPAD` pads the number to six digits with zeros on the left, so a leading zero is never lost.
(2) `CONCAT` puts `19` in front, so `930105` becomes `19930105`.
(3) `STR_TO_DATE` reads that text using the pattern year-month-day (`%Y%m%d`) and returns a real date: 1993-01-05.

We can safely add `19` because the dataset covers 1993 to 1998. The data confirms it: no transaction falls before its account's opening date (Step 4, check 8).

7.3 Problem 2: gender and birth date hidden in one number

The Czech personal number has the format YYMMDD, but **50 is added to the month for women**. Take the example `706213`:

Part	Value	Meaning
Year	70	1970
Month	62	Above 50, so the client is **female**; real month is $62 - 50 = 12$ (December)
Day	13	13th

So the client is a woman born on 13 December 1970. This is a documented convention of the Czech personal number, not a guess, and the data supports it: all 5,369 birth dates decoded successfully.

Main code

```
CASE WHEN (FLOOR(c.birth_number/100) % 100) > 50          (1)
     THEN 'F' ELSE 'M' END AS gender,                     (2)
STR_TO_DATE(CONCAT('19',
  LPAD(FLOOR(c.birth_number/10000), 2, '0'),              (3)
  LPAD(CASE WHEN (FLOOR(c.birth_number/100) % 100) > 50
            THEN (FLOOR(c.birth_number/100) % 100) - 50
            ELSE (FLOOR(c.birth_number/100) % 100)
       END, 2, '0'),                                      (4)
  LPAD(c.birth_number % 100, 2, '0')),                    (5)
  '%Y%m%d') AS birth_date
```

What this code does

1. Cuts out the middle two digits of the number (the month). For `706213` that is `62`.
2. If the month is above 50 the client is female (`F`), otherwise male (`M`).
3. The first two digits are the year: `70`.
4. The month: if it is above 50, subtract 50 to get the real month (62 becomes 12).
5. The last two digits are the day: `13`. All three parts are joined and turned into a real date, as in Problem 1.

7.4 Problem 3: age

We measure age on 1 January 1999 because the data ends in 1998. Using today's date would make everyone more than 25 years older than they were in the data. Ages are then grouped into bands. (Shortened: the full view repeats the age formula in each line.)

Main code

```
TIMESTAMPDIFF(YEAR, p.birth_date, '1999-01-01') AS age,   (1)
CASE WHEN age < 31 THEN '18-30'                           (2)
     WHEN age < 46 THEN '31-45'
     WHEN age < 61 THEN '46-60'
     ELSE '60+' END AS age_band                           (3)
```

What this code does

1. Counts the full years between the birth date and 1 January 1999.
2. Puts each client in the first band that fits: under 31 is `18-30`, under 46 is `31-45`, under 61 is `46-60`.
3. Everybody else is `60+`. Bands make a chart readable: four bars instead of sixty ages.

7.5 Problem 4: Czech codes

Raw value	Readable value
PRIJEM	Credit
VYDAJ	Withdrawal
VYBER	Cash withdrawal
Loan status A	Finished - paid
Loan status B	Finished - not paid
Loan status C	Running - OK
Loan status D	Running - in debt

Main code

```
CASE t.type WHEN 'PRIJEM' THEN 'Credit'                    (1)
            WHEN 'VYDAJ'  THEN 'Withdrawal'
            WHEN 'VYBER'  THEN 'Cash withdrawal'
            ELSE t.type END AS type                        (2)
```

What this code does

(1) Looks at the code in the `type` column and replaces it with the readable word from the table above.

(2) If a code is not in the list, it is kept as it is instead of being lost. The loan status letters are translated the same way in `vw_loan_summary`.

7.6 The views we created

View	What it produces
`vw_client_profile`	`client_id`, gender, birth date and `district_id`, decoded from `birth_number`
`vw_client_profile_full`	Adds age, age band, district name and region
`vw_monthly_transactions`	One row per account, month and type, with total amount and number of transactions
`vw_loan_summary`	Loans with readable status
`vw_account_summary`	One row per account: owner, district, opening date, number of transactions and latest balance
`vw_dq_latest`	The latest data quality results (Step 4)

The table `dim_district` is created at the start of the same script, with the clearer column names from Section 4.

7.7 Two clever pieces of SQL

Monthly summary. Instead of leaving over a million rows for the dashboard, we group them by account, month and type. This gives 368,569 rows and still keeps the monthly picture. (Shortened for reading.)

Main code

```
SELECT t.account_id,                                       (1)
```

```
       DATE_FORMAT(STR_TO_DATE(CONCAT('19', LPAD(t.date,6,'0')),
                   '%Y%m%d'), '%Y-%m-01') AS month,      (2)
       CASE t.type ... END AS type,                      (3)
       SUM(t.amount) AS total_amount,                    (4)
       COUNT(*)      AS txn_count                        (5)
FROM staging.trans t
GROUP BY account_id, month, type;                        (6)
```

What this code does

(1) Keep the account number.
(2) Turn each date into the first day of its month (for example 1993-01-01), so all January transactions share one label.
(3) Translate the Czech type code, as in Problem 4.
(4) Add up the money moved.
(5) Count how many transactions there were.
(6) Make one result row for each account, month and type.

Latest balance of each account. Each transaction stores the balance after it. To find the latest, number each account's transactions from newest to oldest and keep number 1.

Main code

```
WITH last_bal AS (
    SELECT account_id, balance,
           ROW_NUMBER() OVER (PARTITION BY account_id      (1)
                  ORDER BY date DESC, trans_id DESC) AS rn (2)
    FROM staging.trans
)
SELECT a.account_id, b.balance AS latest_balance
FROM staging.account a
LEFT JOIN last_bal b
       ON b.account_id = a.account_id AND b.rn = 1;       (3)
```

What this code does

(1) Look at each account separately and number its transactions 1, 2, 3, and so on.
(2) Newest first. Using `trans_id` as a second sort key breaks ties when two transactions happen on the same day.
(3) Keep only number 1, the newest transaction, and attach its balance to the account. `LEFT JOIN` keeps accounts even if they have no transactions.

Why a bank wants this

One definition for each number. If two analysts decoded gender differently, two reports would disagree and nobody would know which to trust. Views keep the rule in one reviewed place, and business users never see codes like VYDAJ.

8 Step 3: Stored procedures and reporting tables

8.1 The problem with views alone

A view recalculates every time it is used. The monthly summary view groups over a million rows each time, which is slow if Power BI asks for it on every refresh. We want to run the heavy work **once** and keep the result.

8.2 What is a stored procedure?

A **stored procedure** is a named routine saved in the database. You run it with one command, `CALL name()`, and it does a list of steps in order. A view is a saved *question*; a stored procedure is a saved *set of actions*.

Think of it like this

A view is a recipe card. A stored procedure is the cook's whole shift plan: "make soup, then bake bread, then write down how many portions you made". One instruction starts the whole plan.

8.3 The four procedures

Procedure	Job
`sp_refresh_reporting`	Rebuilds the four reporting tables from the views and logs the row counts
`sp_add_dq`	Records one data quality result and decides PASS or FAIL
`sp_run_dq_checks`	Runs all 14 checks
`sp_run_all`	The master switch: refresh first, then run the checks

8.4 Building the reporting tables

For each table, the procedure drops the old one, creates a new one from a view, and writes the row count to `refresh_log`.

Main code

```
DROP TABLE IF EXISTS client_profile;                          (1)
CREATE TABLE client_profile                                   (2)
    AS SELECT * FROM vw_client_profile_full;
INSERT INTO refresh_log (table_name, rows_loaded)             (3)
    SELECT 'client_profile', COUNT(*) FROM client_profile;
```

What this code does

1. Throw away the old table, so a rerun never duplicates rows.
2. Run the view once and save its result as a real table.
3. Write a line in the logbook: which table, and how many rows it now has.

The same three lines are repeated for the other three tables.

Reporting table	Rows	Built from
`dim_district`	77	`staging.district`, renamed columns
`client_profile`	5,369	`vw_client_profile_full`
`account_summary`	4,500	`vw_account_summary`
`monthly_transactions`	368,569	`vw_monthly_transactions`
`loan_summary`	682	`vw_loan_summary`

8.5 Why tables and not just views?

Tables are pre-calculated. Power BI loads them quickly, while a view would recalculate over a million rows each time. We keep the views as the readable "rules" and the tables as the "saved answers". The exception is `vw_dq_latest`: it has only 14 rows, and as a view it always shows the latest run automatically.

8.6 The master procedure

Main code

```
CREATE PROCEDURE sp_run_all()                          (1)
BEGIN
    CALL sp_refresh_reporting();                       (2)
    CALL sp_run_dq_checks();                           (3)
END
```

What this code does

(1) Defines one routine with a single name.

(2) First, rebuild the four reporting tables.

(3) Then run the 14 data quality checks. The order matters: some checks compare the reporting tables with staging, so the tables must exist first.

8.7 The helper that records a check

`sp_add_dq` is called by every check. It is shortened here; the complete version is in Appendix B.

Main code

```
CREATE PROCEDURE sp_add_dq(IN p_run INT, IN p_check VARCHAR(100),
        ..., IN p_failed BIGINT, IN p_total BIGINT,
        IN p_threshold DECIMAL(6,2))                   (1)
BEGIN
    INSERT INTO dq_results (...)
    VALUES (p_run, p_check, ..., p_failed, p_total,
        ROUND(100 * p_failed / NULLIF(p_total, 0), 2),  (2)
        p_threshold,
        CASE WHEN 100 * p_failed / NULLIF(p_total, 0)
                  > p_threshold                        (3)
             THEN 'FAIL' ELSE 'PASS' END);
END
```

What this code does

(1) Takes the facts of one check as inputs: which run, the check name, rows that failed, rows tested and the allowed threshold.

(2) Works out the failure percentage. `NULLIF` avoids a division by zero if there are no rows.

(3) If the failure percentage is above the threshold the status is `FAIL`, otherwise `PASS`. The pass rule lives in this one place for all 14 checks.

8.8 Two helper tables

Table	Content
`refresh_log`	`log_id`, `run_time`, `table_name`, `rows_loaded`: one row for every reporting table rebuilt
`dq_results`	`dq_id`, `run_id`, `run_date`, `check_name`, `table_name`, `failed_rows`, `total_rows`, `failure_pct`, `threshold_pct`, `status`

9 Step 4: Data quality checks

9.1 Goal

Prove with numbers whether the data can be trusted before anyone uses it. The pipeline tests the data on every run, stores the results, and shows them on the dashboard.

9.2 How a check works

1. Count the rows that break a rule (`failed_rows`).
2. Compare with all rows tested to get the failure percentage.
3. Compare with the **threshold**, the allowed percentage. Above it means **FAIL**, otherwise **PASS**.
4. Save the result in `dq_results`, tagged with a run number, so history is kept.

Four of the checks, one from each main group, are shown below.

Main code

```
SELECT COUNT(*) INTO v_failed FROM staging.trans          (1)
WHERE bank IS NULL OR TRIM(bank) = '';                    (2)
CALL sp_add_dq(v_run, 'Blank bank in trans',
               'trans', v_failed, v_total, 5);            (3)

SELECT COUNT(*) - COUNT(DISTINCT trans_id)                (4)
    INTO v_failed FROM staging.trans;

SELECT COUNT(*) INTO v_failed                             (5)
FROM staging.trans t
LEFT JOIN staging.account a ON a.account_id = t.account_id
WHERE a.account_id IS NULL;

SELECT ABS(v_total - COALESCE(SUM(txn_count), 0))         (6)
    INTO v_failed FROM monthly_transactions;
```

What this code does

(1) Completeness: start counting rows and store the count in `v_failed`.
(2) A row counts as a failure if the bank is empty (`NULL`) or only spaces.
(3) Hand the result to the helper, with an allowed threshold of 5%.
(4) Uniqueness: total rows minus the number of different IDs. Any difference means a repeated transaction.
(5) Integrity: join each transaction to its account. If no account is found, the transaction is an orphan.

6. Reconciliation: the transaction count in the monthly table must equal the rows in `trans`. The absolute difference should be zero.

9.3 The 14 checks

No.	Group	Check	Failed	Failure %	Status
1	Completeness	Blank bank in trans	782,812	74.11	FAIL
2	Completeness	Blank counterparty account	760,931	72.04	FAIL
3	Completeness	Blank operation	183,114	17.34	FAIL
4	Completeness	Blank k_symbol	535,314	50.68	FAIL
5	Uniqueness	Duplicate trans_id	0	0.00	PASS
6	Integrity	Orphan transactions (no account)	0	0.00	PASS
7	Integrity	Orphan loans (no account)	0	0.00	PASS
8	Validity	Transaction before account opened	0	0.00	PASS
9	Validity	Negative balance	2,999	0.28	PASS
10	Validity	Birth date could not be decoded	0	0.00	PASS
11	Validity	Implausible client age	0	0.00	PASS
12	Reconciliation	client vs client_profile	0	0.00	PASS
13	Reconciliation	account vs account_summary	0	0.00	PASS
14	Reconciliation	trans vs monthly_transactions	0	0.00	PASS

9.4 The five groups explained simply

Group	Question it asks	What goes wrong if it fails
Completeness	Is anything missing?	Reports built on incomplete data
Uniqueness	Is anything counted twice?	The same money counted twice
Integrity	Does everything link to something real?	A transaction that belongs to no account
Validity	Do the values make sense?	Impossible dates or ages
Reconciliation	Do totals match after each step?	Rows lost or duplicated during transformation

9.5 Why the thresholds differ

- **0%** where any problem is a real defect (duplicates, orphans, mismatched counts).
- **1%** for negative balance. Overdrafts are legitimate in banking, but a high rate would signal a problem.

- **5%** for completeness, where a small share of blanks is normal.

These are judgement calls, which is why the thresholds are visible and easy to change.

9.6 Results in a chart

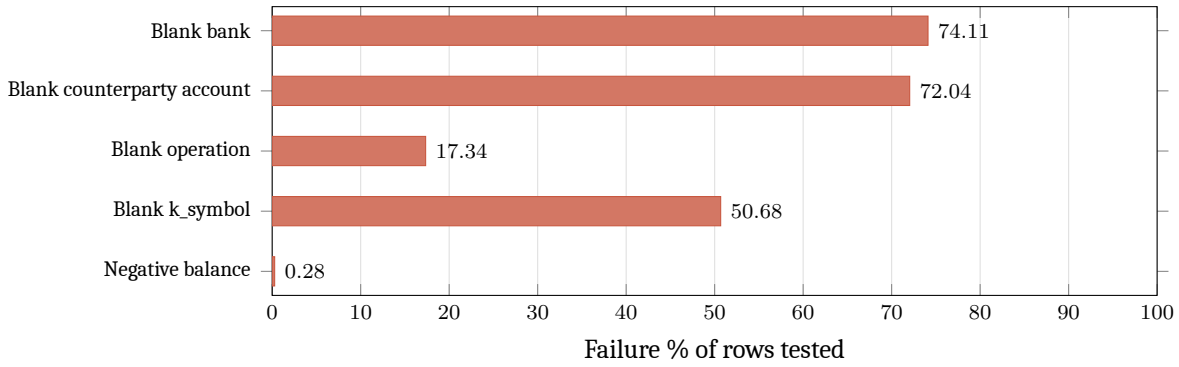


Figure 3: Failure percentages of the four failed completeness checks, plus the negative balance check for comparison. The completeness threshold is 5%, the negative balance threshold is 1%. All other 9 checks had 0 failures.

Why a bank wants this

A dashboard is only as good as its data. These checks catch duplicate transactions, orphan records and lost rows before wrong numbers reach a manager. Banking data standards expect banks to show that risk and reporting data are complete, accurate and reconciled, and this step is a small version of that.

10 Step 5: Root cause analysis

10.1 Why it is needed

A failed check only says "something is off". Root cause analysis answers: **why**? There are two possible answers:

- **Error:** the data is wrong or lost, and should be fixed.
- **Structural:** the field does not apply to those rows, so the check is too strict.

Getting this wrong has a cost either way. Calling a real defect "normal" lets bad data through. Calling a normal blank a defect wastes the team's time.

10.2 The method: three tests

A blank is called **structural** when all three are true:

1. **Concentrated:** one group of rows is blank almost 100% of the time and another group almost 0%, instead of blanks being scattered.
2. **Consistent:** the same value repeats (all `0`, all `UROK`). That points to a system rule, not human mistakes.

3. **Makes business sense:** the field really cannot apply to that kind of transaction.

A real error looks the opposite: rare, scattered and unexplained.

In practice we group the rows by a meaningful column, usually `operation`, and compare the blank rate in each group.

Main code

```
SELECT operation,                                          (1)
       COUNT(*) AS txns,                                   (2)
       ROUND(100 * SUM(bank IS NULL OR TRIM(bank) = '')
             / COUNT(*), 1) AS pct_blank_bank              (3)
FROM staging.trans
GROUP BY operation                                         (4)
ORDER BY txns DESC;
```

What this code does

(1) Show one result row for each kind of operation (cash withdrawal, cash deposit, transfer, and so on).

(2) How many transactions that operation has.

(3) What percentage of them have a blank bank. `SUM(condition)` adds up 1 for each blank row.

(4) Split the table into those groups. If the blank rate is near 100% in one group and near 0% in another, the blanks are structural, not random.

10.3 Finding 1: blank bank and blank counterparty account

Group	Rows	Blank bank
Transfers (PREVOD NA UCET, PREVOD Z UCTU)	273,509	1
All other operations (cash, card, no operation)	782,811	100%

The bank and the counterparty account exist only when money moves to or from another bank. A cash withdrawal at the counter has no other bank. The blanks sit exactly where there is nothing to fill in, so they are **structural**.

10.4 Finding 2: blank operation

All 183,114 blank-operation rows are credits with `k_symbol` `UROK`, which means interest that the bank adds itself. No customer action happens, so there is no operation to record. **Structural.**

The query behind this finding:

Main code

```
SELECT k_symbol, COUNT(*) AS txns                          (1)
FROM staging.trans
WHERE operation IS NULL OR TRIM(operation) = ''            (2)
GROUP BY k_symbol;                                         (3)
```

What this code does

(1) Count transactions for each purpose code.

(2) But only among rows whose operation is blank.

(3) The answer comes back as a single group, `UROK`, with 183,114 rows: one repeated value, which is the signature of a system rule.

10.5 Finding 3: blank k_symbol

The `k_symbol` is a payment purpose code. Where blank:

Operation	Rows with blank k_symbol
Cash withdrawal (type VYDAJ)	258,009
Cash deposit (VKLAD)	156,743
Transfer out (PREVOD NA UCET)	60,972
Transfer in (PREVOD Z UCTU)	34,888
Cash withdrawal (type VYBER)	16,666
Card withdrawal (VYBER KARTOU)	8,036

82.1% of the blank rows are cash or card movements, which carry no purpose code. The other 95,860 are transfers, which is 35.0% of all transfers, where the sender gave no purpose. So `k_symbol` is an **optional field**, and blank means "no specific purpose". **Structural.**

10.6 Finding 4: negative balances (a passed check, reviewed anyway)

There are 2,999 negative-balance rows (0.28%), and they come from only **288 of 4,500 accounts**, about 10 rows each. Repeated negatives in a small group of accounts fit overdrafts, not scattered data errors.

The query behind this finding:

Main code

```
SELECT COUNT(DISTINCT account_id)                    (1)
           AS accounts_with_negative_balance,
       COUNT(*) AS negative_rows                     (2)
FROM staging.trans
WHERE balance < 0;                                   (3)
```

What this code does

1. How many *different* accounts ever went negative: 288.
2. How many transaction rows show a negative balance: 2,999.
3. Look only at rows where the balance is below zero. 2,999 rows over 288 accounts is about 10 rows per account, so the problem is concentrated, not scattered.

10.7 Other discoveries

Discovery	Type
The source writes `0` to mean "no account" on 21,881 non-transfer rows (13,845 cash withdrawals and all 8,036 card withdrawals) instead of leaving it empty. The completeness check counts only empty values, so it *understates* missing accounts.	Source-system issue
One outgoing transfer has a blank bank and account.	Real defect (1 in 273,509)
Cash withdrawals are coded under two type values: VYDAJ (418,252 rows) and VYBER (16,666 rows). The same event is recorded in two ways.	Source-system issue
Interest credits have no operation recorded.	Source-system issue

10.8 Recommendations

1. Measure blank bank and account only within transfers, where the expected rate is close to zero.
2. Treat account $= 0$ as missing and label those rows "No counterparty".
3. Label rows with blank operation and `UROK` as "Interest credit".
4. Treat `k_symbol` as optional, or check it only where a purpose is expected.
5. Standardise the type value of cash withdrawals.
6. Raise the placeholder `0`, the double coding and the single defective transfer with the source-system team.

10.9 How certain are these conclusions?

- **The counts are exact.** Every row was tested, so there is no sampling error.
- **The interpretation is a judgement.** Calling the failures "structural" rests on the three tests and on business sense. A data owner at the bank should confirm it, which is why the note says "recommend", not "proved".

Why a bank wants this

An analyst who only reports "4 checks failed" creates panic or gets ignored. One who explains that three are structural, that one source system writes `0`, and that one transfer is a real defect gives the business something to act on.

11 Step 6: Automation

11.1 Goal

Make the whole pipeline run with one command, so it can be repeated by anyone and scheduled to repeat on its own.

11.2 What runs, in order

The `main()` function in `run_pipeline.py` is the control centre.

Main code

```
def main():
    load_staging()                                          1
    build_indexes()                                         2
    if not procedures_exist():                              3
        print("Now run sql/01_setup.sql, then run again.")
        return
    refresh_reporting()                                     4
    print("Pipeline finished successfully")                 5
```

What this code does

1. Load the six files into `staging` (Step 1).
2. Add the indexes.
3. Check that the stored procedures exist. They do after the setup script has been run once. If not, tell the user what to do and stop.
4. Rebuild `dim_district`, then run `CALL reporting.sp_run_all()`, which rebuilds the reporting tables and runs the 14 checks (Steps 3 and 4).
5. Print the success message. If anything fails, the script prints "Pipeline FAILED" with the reason and exits with an error code, which a scheduler can detect.

11.3 First-time setup versus every later run

Order	Action
1	Put the six CSV files in the `data` folder
2	Run `00_create_databases.sql` in MySQL Workbench
3	In Command Prompt: `pip install -r requirements.txt`
4	Set the password for this window: `set MYSQL_PASSWORD=...`
5	Run `python python\run_pipeline.py`. It loads staging and asks for the setup script
6	Run `01_setup.sql` in MySQL Workbench
7	Run `python python\run_pipeline.py` again. It ends with "Pipeline finished successfully"

Every later run needs only step 7.

11.4 Logging at every level

Log	Records
`staging.run_log`	Each table load: rows, success or fail, error text
`reporting.refresh_log`	Each reporting table rebuilt, with row count
`reporting.dq_results`	Every data quality check of every run
`pipeline.log`	What the Python script did (created automatically)

If a run fails, the log shows where.

11.5 Scheduling

The pipeline can be run on a schedule with Windows Task Scheduler, using a small `.bat` file that changes to the project folder and runs the script. A scheduled task does not see a password typed into one Command Prompt window, so the password would be stored once as a permanent environment variable with the `setx` command.

Main code

```
@echo off                                                    (1)
cd /d C:\path\to\banking-etl-pipeline                        (2)
python python\run_pipeline.py >> pipeline_run.log 2>&1       (3)
```

What this code does

(1) Do not print each command on screen.
(2) Go to the project folder.
(3) Run the pipeline and *append* everything it prints to `pipeline_run.log`, so a scheduled run leaves a record even when nobody is watching.

Practical notes:

- Do not query the database while the pipeline runs. A heavy query running at the same time caused a database deadlock during testing.
- The computer must be on at the scheduled time.
- Power BI Desktop does not refresh by itself. After the pipeline runs, press **Refresh**. Automatic refresh in the Power BI Service would need a gateway and a paid plan.

Why a bank wants this

The same result every day, whoever is on shift. No manual steps, so no manual errors. Failures are caught and recorded instead of being discovered by a manager who spots a wrong number. This is the difference between a one-off analysis and a production process.

12 Step 7: The Power BI dashboard

12.1 Goal

Show the clean data to the business on a few clear pages, so a manager can answer questions in seconds without writing SQL.

12.2 Connecting to the data

Power BI connects to MySQL with **Get data → MySQL database** (this needs the MySQL Connector/NET driver installed). It reads **only the `reporting` database**, never `staging`. Six objects were imported and renamed in Power Query to clean names:

Imported object	Used for
`client_profile`	Clients: age, gender, region
`account_summary`	Accounts: balances, owner, district
`monthly_transactions`	Monthly money in and out
`loan_summary`	Loans and their status
`dim_district`	District and region names
`vw_dq_latest`	Latest data quality results

12.3 The data model (relationships)

Tables are linked through shared ID columns, so clicking something in one table filters the others.

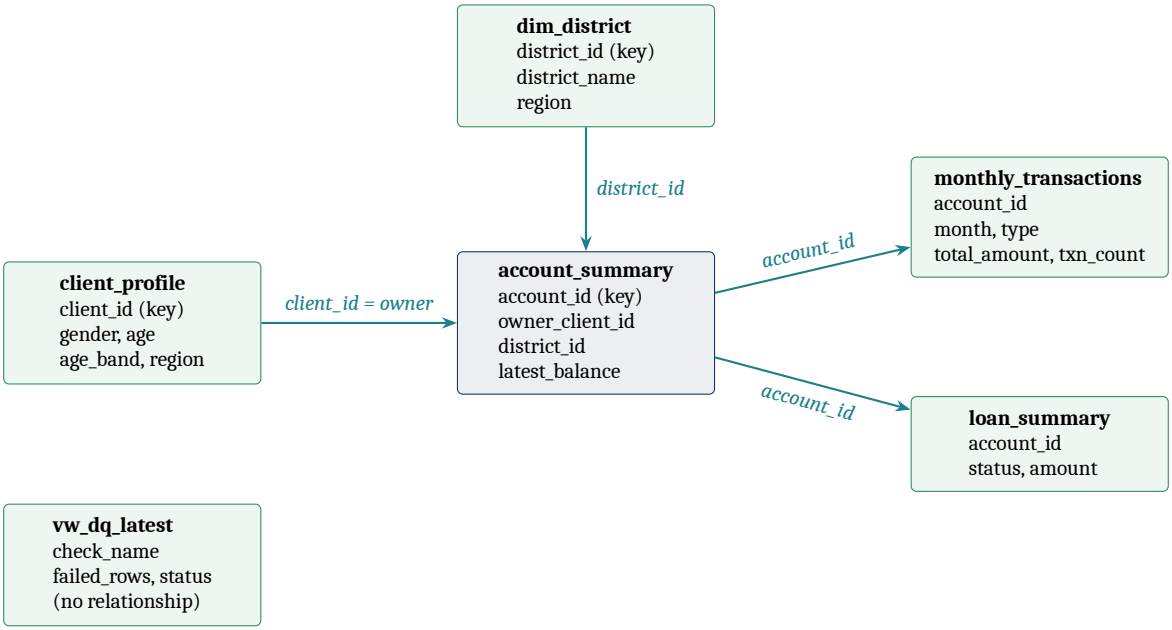


Figure 4: The Power BI data model. All four relationships are one-to-many with a single filter direction. `account_summary` is the hub.

12.4 The 12 measures

A **measure** is a DAX formula that recalculates for whatever the page is currently filtered to. Click a region and every measure re-answers for that region.

Main code

```
Total Transaction Volume = SUM(monthly_transactions[total_amount])                    1
Credits = CALCULATE(SUM(monthly_transactions[total_amount]),
          monthly_transactions[type] = "Credit")                                      2
Debits = CALCULATE(SUM(monthly_transactions[total_amount]),
   monthly_transactions[type] IN {"Withdrawal", "Cash withdrawal"})                   3
Net Flow = [Credits] - [Debits]                                                       4
Txn Count = SUM(monthly_transactions[txn_count])                                      5
Active Accounts = DISTINCTCOUNT(monthly_transactions[account_id])                     6
Total Clients = DISTINCTCOUNT(client_profile[client_id])                              7
Total Balance = SUM(account_summary[latest_balance])                                  8
Avg Balance = AVERAGE(account_summary[latest_balance])                                9
Loans In Trouble % = DIVIDE(CALCULATE(COUNTROWS(loan_summary),
    loan_summary[status] IN {"Finished - not paid", "Running - in debt"}),
    COUNTROWS(loan_summary))                                                          10
```

```
DQ Pass Rate = DIVIDE(CALCULATE(COUNTROWS(vw_dq_latest),
    vw_dq_latest[status] = "PASS"), COUNTROWS(vw_dq_latest))          11
DQ Failed Checks = CALCULATE(COUNTROWS(vw_dq_latest),
    vw_dq_latest[status] = "FAIL")                                    12
```

What this code does

1. All money moved, added up.
2. Money coming in: the total, but only for rows of type Credit.
3. Money going out: Withdrawal plus Cash withdrawal. Both are needed because the source codes cash withdrawals under two type values (Section 10); using only one would understate money going out.
4. Credits minus Debits: the net movement of money.
5. The number of transactions.
6. How many different accounts had activity. `DISTINCTCOUNT` counts each account once.
7. How many different clients.
8. The sum of every account's latest balance.
9. The average account's latest balance.
10. Loans that are "Finished - not paid" or "Running - in debt", divided by all loans. `DIVIDE` returns blank instead of an error if there are no loans.
11. The share of checks with status PASS in the latest run.
12. How many checks have status FAIL in the latest run.

12.5 Page 1: Overview

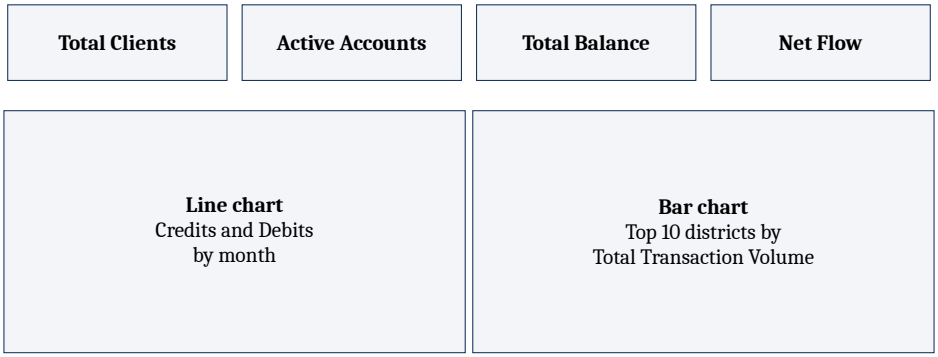


Figure 5: Layout of page 1 (a sketch of the design, not a screenshot).

- **Cards:** Total Clients 5,369; Active Accounts 4,500; Total Balance about 197.14 million; Net Flow about 197.15 million. Power BI shortens large numbers by default (5,369 shows as "5K"), so the card's display units are set to None to show the full figure.
- **Line chart:** Credits and Debits by month, 1993 to 1998. Both lines rise over the period, with Credits above Debits. The monthly field is used, not the automatic Year–Quarter–Month–Day hierarchy, which would show only years.
- **Bar chart:** the ten districts with the largest transaction volume, using a Top N filter. Prague is far ahead of the rest.

Net Flow and Total Balance are almost equal. This is expected: each account's final balance is its credits minus its debits over the full history, so summing both gives nearly the same total.

12.6 Page 2: Customers and Loans

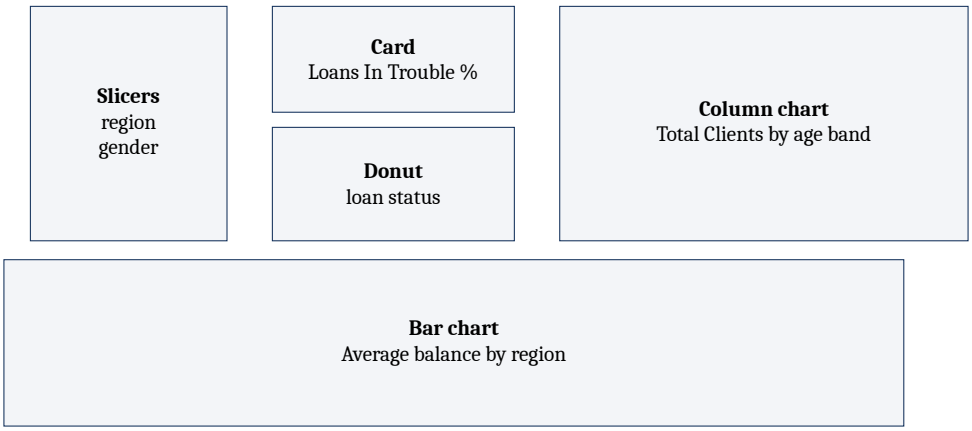


Figure 6: Layout of page 2 (a sketch of the design, not a screenshot).

- **Slicers** for region and gender filter every visual on the page.
- The **donut chart** shows how loans split between finished-paid, finished-not-paid, running-OK and running-in-debt. It counts loans (the count of `loan_id`), not the sum of the IDs.
- The **column chart** counts clients in the four age bands.
- The **bar chart** compares average balance by region, using the region where the client lives.

Because of the relationships in the model, choosing a region in the slicer also changes the loan and balance numbers.

12.7 Page 3: Data Quality

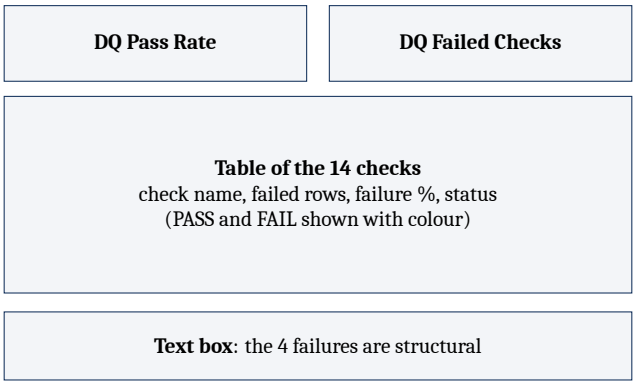


Figure 7: Layout of page 3 (a sketch of the design, not a screenshot).

The page reads from `vw_dq_latest`, so it always shows the most recent run. The pass rate is 10 of 14 checks, and the failed-check count is 4. A text box explains that the four failures are structural, with the evidence in the root cause note. In the table, the aggregation of numeric columns is set to *Don't summarize*, so each row shows the exact value of one check.

Why a bank wants this

- **Speed:** managers see loan risk or regional balances without waiting for an analyst.
- **One version of the truth:** everybody looks at the same tables.
- **Interactive:** slicers let them drill into a region or age group themselves.
- **Visible data quality:** most dashboards hide data problems. Page 3 shows them, which builds trust.

13 What we created and what we kept

13.1 Everything in the project, in order of creation

Step	Name	Type	Database	Purpose
1	account	table	staging	Raw accounts
1	client	table	staging	Raw clients
1	disp	table	staging	Raw client–account links
1	district	table	staging	Raw district facts
1	loan	table	staging	Raw loans
1	trans	table	staging	Raw transactions
1	run_log	table	staging	Records each load
2	dim_district	table	reporting	Renamed district columns
2	vw_client_profile	view	reporting	Decodes gender and birth date
2	vw_client_profile_full	view	reporting	Adds age, band, region
2	vw_monthly_transactions	view	reporting	Monthly totals with readable types
2	vw_loan_summary	view	reporting	Readable loan status
2	vw_account_summary	view	reporting	Latest balance and owner
3	client_profile	table	reporting	Saved clients for the dashboard
3	account_summary	table	reporting	Saved accounts for the dashboard
3	monthly_transactions	table	reporting	Saved monthly totals
3	loan_summary	table	reporting	Saved loans
3	refresh_log	table	reporting	Records each rebuild
3	sp_refresh_reporting	procedure	reporting	Rebuilds the four tables
3	sp_run_all	procedure	reporting	Runs refresh, then checks
4	dq_results	table	reporting	History of every check
4	vw_dq_latest	view	reporting	Latest run only
4	sp_add_dq	procedure	reporting	Writes one check result
4	sp_run_dq_checks	procedure	reporting	Runs the 14 checks

13.2 What each layer is for

In one line

staging = raw copy. **views** = the cleaning rules. **reporting tables** = the saved answers. **dq_results** = the proof the data was checked. **Power BI** = the display.

13.3 Why each layer is kept

Layer	Reason it is kept
Raw `staging` tables	The original must survive. Any rule can be corrected and the data rebuilt
Views	One readable, reviewable definition of each business rule
Reporting tables	Pre-calculated, so the dashboard is fast
Logs	Proof of what ran, when, and with how many rows
`dq_results` history	Quality can be compared run after run

13.4 Why `refresh_log` and `run_log` are different

`run_log` (in `staging`) records the **load** of each source file, written by Python. `refresh_log` (in `reporting`) records the **rebuild** of each reporting table, written by a stored procedure. Together with `dq_results` they answer three questions: did the files load, did the clean tables rebuild, and can the data be trusted?

13.5 Views and tables in Power BI

To Power BI a view looks and works like a table, so it can import either. We import real tables for the heavy objects because a view would regroup over a million rows each time. We import `vw_dq_latest` as a view because it has 14 rows and must show only the latest run.

14 Results, limitations and lessons

14.1 What was achieved

- 1,056,320 transactions, 5,369 clients, 4,500 accounts and 682 loans loaded and reconciled across every step with zero row differences.
- Six SQL views and four stored procedures that rebuild the reporting layer on demand.
- Fourteen automated checks. Ten pass. Four completeness checks fail, and root cause analysis shows they are structural.
- Four findings for the source-system team: the `0` placeholder, the double coding of cash withdrawals, interest rows without an operation, and one defective transfer.
- A three-page Power BI dashboard whose last page shows the data quality status.

14.2 Limitations (stated honestly)

- The Berka data is from 1993 to 1998. It is good for learning the method, but its insights are not current business advice.
- The root cause conclusions are judgements backed by exact counts, and should be confirmed by a data owner.
- The pipeline is run manually or by an optional local schedule. A production setup would use an enterprise scheduler.

- Power BI Desktop refreshes manually. Automatic refresh needs the Power BI Service and a gateway.
- Check thresholds (0%, 1%, 5%) are judgement calls, kept visible so they can be changed.
- The blank-bank and blank-account checks could be narrowed to transfers only, which would remove two of the four failures.

14.3 Lessons learned

1. Keep raw data separate from clean data.
2. Put each business rule in one place.
3. A failed check is a question, not a verdict. Always ask why.
4. Log everything: a partial load was caught only because row counts were recorded.
5. Never put passwords in code.
6. Let the dashboard show the data quality, not hide it.

A Glossary

Term	Meaning
CSV	A plain text file where values are separated by a character (here a semicolon)
ETL / ELT	Extract, Transform, Load, or Extract, Load, Transform (as in this project)
Staging	The area that holds raw data exactly as loaded
View	A saved SQL query that behaves like a table but stores no data
Stored procedure	A named set of SQL steps saved in the database and run with `CALL`
Index	A lookup structure that makes searching a large table fast
Chunk	A small part of a large file, read and loaded one piece at a time
Reconciliation	Checking that totals match between two steps
Orphan record	A record that points to something that does not exist
Threshold	The allowed share of bad rows before a check fails
Structural blank	A blank that is expected because the field does not apply
Root cause analysis	Finding out why a problem happened
Measure (DAX)	A formula in Power BI that recalculates for the current filter
Slicer	A Power BI filter box the viewer can click
Data lineage	The record of where each number came from

B Full code listings

This appendix contains the complete source code of the project. The database password is not stored anywhere in the code.

B.1 sql/00_create_databases.sql

```
CREATE DATABASE IF NOT EXISTS staging;
CREATE DATABASE IF NOT EXISTS reporting;

CREATE TABLE IF NOT EXISTS staging.run_log (
    run_id        INT AUTO_INCREMENT PRIMARY KEY,
    run_time      DATETIME DEFAULT CURRENT_TIMESTAMP,
    table_name    VARCHAR(50),
    rows_loaded   INT,
    status        VARCHAR(10),
    error_message TEXT
);
```

B.2 sql/01_setup.sql

```
USE reporting;

DROP TABLE IF EXISTS dim_district;

CREATE TABLE dim_district AS
SELECT
    A1  AS district_id,
    A2  AS district_name,
    A3  AS region,
    A4  AS population,
    A9  AS num_cities,
    A10 AS urban_ratio_pct,
    A11 AS avg_salary,
    A12 AS unemployment_rate_1995,
    A13 AS unemployment_rate_1996,
    A14 AS entrepreneurs_per_1000,
    A15 AS crimes_1995,
    A16 AS crimes_1996
FROM staging.district;

CREATE OR REPLACE VIEW vw_client_profile AS
SELECT
    c.client_id,
    CASE WHEN (FLOOR(c.birth_number/100) % 100) > 50 THEN 'F' ELSE 'M' END AS gender,
    STR_TO_DATE(CONCAT('19',
        LPAD(FLOOR(c.birth_number/10000), 2, '0'),
        LPAD(CASE WHEN (FLOOR(c.birth_number/100) % 100) > 50
                  THEN (FLOOR(c.birth_number/100) % 100) - 50
                  ELSE (FLOOR(c.birth_number/100) % 100) END, 2, '0'),
        LPAD(c.birth_number % 100, 2, '0')), '%Y%m%d') AS birth_date,
    c.district_id
FROM staging.client c;

CREATE OR REPLACE VIEW vw_client_profile_full AS
SELECT
    p.client_id, p.gender, p.birth_date,
    TIMESTAMPDIFF(YEAR, p.birth_date, '1999-01-01') AS age,
    CASE WHEN TIMESTAMPDIFF(YEAR, p.birth_date, '1999-01-01') < 31 THEN '18-30'
         WHEN TIMESTAMPDIFF(YEAR, p.birth_date, '1999-01-01') < 46 THEN '31-45'
         WHEN TIMESTAMPDIFF(YEAR, p.birth_date, '1999-01-01') < 61 THEN '46-60'
         ELSE '60+' END AS age_band,
    d.district_name, d.region
FROM vw_client_profile p
LEFT JOIN dim_district d ON d.district_id = p.district_id;

CREATE OR REPLACE VIEW vw_monthly_transactions AS
```

```
SELECT
    t.account_id,
    DATE_FORMAT(STR_TO_DATE(CONCAT('19', LPAD(t.date,6,'0')), '%Y%m%d'), '%Y-%m-01') AS month,
    CASE t.type WHEN 'PRIJEM' THEN 'Credit'
                WHEN 'VYDAJ'  THEN 'Withdrawal'
                WHEN 'VYBER'  THEN 'Cash withdrawal'
                ELSE t.type END AS type,
    SUM(t.amount) AS total_amount,
    COUNT(*)      AS txn_count
FROM staging.trans t
GROUP BY t.account_id,
         DATE_FORMAT(STR_TO_DATE(CONCAT('19', LPAD(t.date,6,'0')), '%Y%m%d'), '%Y-%m-01'),
         CASE t.type WHEN 'PRIJEM' THEN 'Credit'
                     WHEN 'VYDAJ'  THEN 'Withdrawal'
                     WHEN 'VYBER'  THEN 'Cash withdrawal'
                     ELSE t.type END;

CREATE OR REPLACE VIEW vw_loan_summary AS
SELECT
    l.loan_id, l.account_id,
    STR_TO_DATE(CONCAT('19', LPAD(l.date,6,'0')), '%Y%m%d') AS loan_date,
    l.amount, l.duration, l.payments,
    CASE l.status WHEN 'A' THEN 'Finished - paid'
                  WHEN 'B' THEN 'Finished - not paid'
                  WHEN 'C' THEN 'Running - OK'
                  WHEN 'D' THEN 'Running - in debt' END AS status
FROM staging.loan l;

CREATE OR REPLACE VIEW vw_account_summary AS
WITH last_bal AS (
    SELECT account_id, balance,
           ROW_NUMBER() OVER (PARTITION BY account_id ORDER BY date DESC, trans_id DESC) AS rn
    FROM staging.trans
),
cnt AS (
    SELECT account_id, COUNT(*) AS txn_count FROM staging.trans GROUP BY account_id
)
SELECT
    a.account_id,
    d.client_id AS owner_client_id,
    a.district_id,
    a.frequency,
    STR_TO_DATE(CONCAT('19', LPAD(a.date,6,'0')), '%Y%m%d') AS open_date,
    c.txn_count,
    b.balance AS latest_balance
FROM staging.account a
LEFT JOIN staging.disp d ON d.account_id = a.account_id AND d.type = 'OWNER'
LEFT JOIN cnt c ON c.account_id = a.account_id
LEFT JOIN last_bal b ON b.account_id = a.account_id AND b.rn = 1;

CREATE TABLE IF NOT EXISTS refresh_log (
    log_id      INT AUTO_INCREMENT PRIMARY KEY,
    run_time    DATETIME DEFAULT CURRENT_TIMESTAMP,
    table_name  VARCHAR(50),
    rows_loaded INT
);

CREATE TABLE IF NOT EXISTS dq_results (
    dq_id         INT AUTO_INCREMENT PRIMARY KEY,
    run_id        INT,
    run_date      DATETIME DEFAULT CURRENT_TIMESTAMP,
    check_name    VARCHAR(100),
    table_name    VARCHAR(50),
    failed_rows   BIGINT,
    total_rows    BIGINT,
    failure_pct   DECIMAL(6,2),
    threshold_pct DECIMAL(6,2),
    status        VARCHAR(4)
);

CREATE OR REPLACE VIEW vw_dq_latest AS
SELECT * FROM dq_results WHERE run_id = (SELECT MAX(run_id) FROM dq_results);

DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS sp_refresh_reporting$$
CREATE PROCEDURE sp_refresh_reporting()
BEGIN
    DROP TABLE IF EXISTS client_profile;
    CREATE TABLE client_profile AS SELECT * FROM vw_client_profile_full;
    INSERT INTO refresh_log (table_name, rows_loaded) SELECT 'client_profile', COUNT(*) FROM client_profile;

    DROP TABLE IF EXISTS account_summary;
    CREATE TABLE account_summary AS SELECT * FROM vw_account_summary;
    INSERT INTO refresh_log (table_name, rows_loaded) SELECT 'account_summary', COUNT(*) FROM account_summary;

    DROP TABLE IF EXISTS monthly_transactions;
    CREATE TABLE monthly_transactions AS
        SELECT account_id, CAST(month AS DATE) AS month, type, total_amount, txn_count
        FROM vw_monthly_transactions;
    INSERT INTO refresh_log (table_name, rows_loaded) SELECT 'monthly_transactions', COUNT(*) FROM
     monthly_transactions;

    DROP TABLE IF EXISTS loan_summary;
    CREATE TABLE loan_summary AS SELECT * FROM vw_loan_summary;
    INSERT INTO refresh_log (table_name, rows_loaded) SELECT 'loan_summary', COUNT(*) FROM loan_summary;
END$$

DROP PROCEDURE IF EXISTS sp_add_dq$$
CREATE PROCEDURE sp_add_dq(IN p_run INT, IN p_check VARCHAR(100), IN p_table VARCHAR(50),
                           IN p_failed BIGINT, IN p_total BIGINT, IN p_threshold DECIMAL(6,2))
BEGIN
    INSERT INTO dq_results (run_id, check_name, table_name, failed_rows, total_rows,
                            failure_pct, threshold_pct, status)
    VALUES (p_run, p_check, p_table, p_failed, p_total,
            ROUND(100 * p_failed / NULLIF(p_total, 0), 2), p_threshold,
            CASE WHEN 100 * p_failed / NULLIF(p_total, 0) > p_threshold THEN 'FAIL' ELSE 'PASS' END);
END$$

DROP PROCEDURE IF EXISTS sp_run_dq_checks$$
CREATE PROCEDURE sp_run_dq_checks()
BEGIN
    DECLARE v_run INT;
    DECLARE v_total BIGINT;
    DECLARE v_failed BIGINT;

    SELECT COALESCE(MAX(run_id), 0) + 1 INTO v_run FROM dq_results;

    SELECT COUNT(*) INTO v_total FROM staging.trans;

    SELECT COUNT(*) INTO v_failed FROM staging.trans WHERE bank IS NULL OR TRIM(bank) = '';
    CALL sp_add_dq(v_run, 'Blank bank in trans', 'trans', v_failed, v_total, 5);

    SELECT COUNT(*) INTO v_failed FROM staging.trans WHERE account IS NULL OR TRIM(account) = '';
    CALL sp_add_dq(v_run, 'Blank counterparty account in trans', 'trans', v_failed, v_total, 5);

    SELECT COUNT(*) INTO v_failed FROM staging.trans WHERE operation IS NULL OR TRIM(operation) = '';
    CALL sp_add_dq(v_run, 'Blank operation in trans', 'trans', v_failed, v_total, 5);

    SELECT COUNT(*) INTO v_failed FROM staging.trans WHERE k_symbol IS NULL OR TRIM(k_symbol) = '';
    CALL sp_add_dq(v_run, 'Blank k_symbol in trans', 'trans', v_failed, v_total, 5);

    SELECT COUNT(*) - COUNT(DISTINCT trans_id) INTO v_failed FROM staging.trans;
    CALL sp_add_dq(v_run, 'Duplicate trans_id', 'trans', v_failed, v_total, 0);

    SELECT COUNT(*) INTO v_failed
    FROM staging.trans t LEFT JOIN staging.account a ON a.account_id = t.account_id
    WHERE a.account_id IS NULL;
    CALL sp_add_dq(v_run, 'Orphan transactions (no account)', 'trans', v_failed, v_total, 0);

    SELECT COUNT(*) INTO v_total FROM staging.loan;
    SELECT COUNT(*) INTO v_failed
    FROM staging.loan l LEFT JOIN staging.account a ON a.account_id = l.account_id
    WHERE a.account_id IS NULL;
    CALL sp_add_dq(v_run, 'Orphan loans (no account)', 'loan', v_failed, v_total, 0);

    SELECT COUNT(*) INTO v_total FROM staging.trans;
    SELECT COUNT(*) INTO v_failed
```

```
    FROM staging.trans t JOIN staging.account a ON a.account_id = t.account_id
    WHERE t.date < a.date;
    CALL sp_add_dq(v_run, 'Transaction before account open date', 'trans', v_failed, v_total, 0);

    SELECT COUNT(*) INTO v_failed FROM staging.trans WHERE balance < 0;
    CALL sp_add_dq(v_run, 'Negative balance', 'trans', v_failed, v_total, 1);

    SELECT COUNT(*) INTO v_total FROM staging.client;
    SELECT COUNT(*) INTO v_failed FROM vw_client_profile WHERE birth_date IS NULL;
    CALL sp_add_dq(v_run, 'Birth date could not be decoded', 'client', v_failed, v_total, 0);

    SELECT COUNT(*) INTO v_total FROM client_profile;
    SELECT COUNT(*) INTO v_failed FROM client_profile WHERE age < 10 OR age > 100;
    CALL sp_add_dq(v_run, 'Implausible client age', 'client_profile', v_failed, v_total, 0);

    SELECT COUNT(*) INTO v_total FROM staging.client;
    SELECT ABS(v_total - COUNT(*)) INTO v_failed FROM client_profile;
    CALL sp_add_dq(v_run, 'Row count: client vs client_profile', 'client_profile', v_failed, v_total, 0);

    SELECT COUNT(*) INTO v_total FROM staging.account;
    SELECT ABS(v_total - COUNT(*)) INTO v_failed FROM account_summary;
    CALL sp_add_dq(v_run, 'Row count: account vs account_summary', 'account_summary', v_failed, v_total, 0);

    SELECT COUNT(*) INTO v_total FROM staging.trans;
    SELECT ABS(v_total - COALESCE(SUM(txn_count), 0)) INTO v_failed FROM monthly_transactions;
    CALL sp_add_dq(v_run, 'Row count: trans vs monthly_transactions', 'monthly_transactions', v_failed,
     v_total, 0);
END$$

DROP PROCEDURE IF EXISTS sp_run_all$$
CREATE PROCEDURE sp_run_all()
BEGIN
    CALL sp_refresh_reporting();
    CALL sp_run_dq_checks();
END$$

DELIMITER ;
```

B.3 sql/02_root_cause.sql

```
SELECT operation, type,
       COUNT(*) AS txns,
       SUM(bank IS NULL OR TRIM(bank) = '')       AS blank_bank,
       SUM(account IS NULL OR TRIM(account) = '') AS blank_account
FROM staging.trans
GROUP BY operation, type
ORDER BY txns DESC;

SELECT operation,
       COUNT(*) AS txns,
       ROUND(100 * SUM(bank IS NULL OR TRIM(bank) = '') / COUNT(*), 1) AS pct_blank_bank
FROM staging.trans
GROUP BY operation
ORDER BY txns DESC;

SELECT account, COUNT(*) AS txns
FROM staging.trans
WHERE operation = 'VYBER KARTOU'
GROUP BY account ORDER BY txns DESC LIMIT 5;

SELECT account, COUNT(*) AS txns
FROM staging.trans
WHERE operation = 'VYBER' AND type = 'VYDAJ'
  AND NOT (account IS NULL OR TRIM(account) = '')
GROUP BY account ORDER BY txns DESC LIMIT 5;

SELECT k_symbol, COUNT(*) AS txns
FROM staging.trans
WHERE operation IS NULL OR TRIM(operation) = ''
GROUP BY k_symbol ORDER BY txns DESC;
```

```sql
SELECT 1900 + FLOOR(date / 10000) AS yr,
       COUNT(*) AS txns,
       ROUND(100 * SUM(operation IS NULL OR TRIM(operation) = '') / COUNT(*), 1) AS pct_blank_operation
FROM staging.trans
GROUP BY yr ORDER BY yr;

SELECT operation, type, COUNT(*) AS txns
FROM staging.trans
WHERE k_symbol IS NULL OR TRIM(k_symbol) = ''
GROUP BY operation, type
ORDER BY txns DESC;

SELECT COUNT(DISTINCT account_id) AS accounts_with_negative_balance,
       COUNT(*) AS negative_rows
FROM staging.trans
WHERE balance < 0;

SELECT account_id, COUNT(*) AS negative_rows, MIN(balance) AS lowest_balance
FROM staging.trans
WHERE balance < 0
GROUP BY account_id
ORDER BY negative_rows DESC
LIMIT 10;

SELECT operation, type, COUNT(*) AS txns
FROM staging.trans
WHERE operation = 'VYBER'
GROUP BY operation, type;
```

B.4 sql/03_data_dictionary.sql

```sql
SELECT table_name, column_name, data_type, ordinal_position
FROM information_schema.columns
WHERE table_schema = 'reporting'
  AND table_name IN ('client_profile', 'account_summary', 'monthly_transactions',
                     'loan_summary', 'dim_district', 'dq_results', 'refresh_log')
ORDER BY table_name, ordinal_position;
```

B.5 python/run_pipeline.py

```python
import os
import sys
import logging
from pathlib import Path

import pandas as pd
from sqlalchemy import create_engine, text
from sqlalchemy.engine import URL

BASE = Path(__file__).resolve().parent.parent
DATA = BASE / "data"
TABLES = ["account", "client", "disp", "district", "loan", "trans"]
INDEXES = [("account", "account_id"), ("client", "client_id"), ("disp", "account_id"),
           ("disp", "client_id"), ("loan", "account_id"), ("trans", "account_id"),
           ("trans", "trans_id")]

logging.basicConfig(filename=BASE / "pipeline.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
logger = logging.getLogger("pipeline")

password = os.environ.get("MYSQL_PASSWORD")
if not password:
    sys.exit("Set the MYSQL_PASSWORD environment variable first.")

engine = create_engine(URL.create("mysql+pymysql", username="root", password=password,
                                  host=os.environ.get("MYSQL_HOST", "localhost"),
                                  port=3306, database="staging"))
```

```
def find_file(name):
    for ext in (".csv", ".asc"):
        p = DATA / f"{name}{ext}"
        if p.exists():
            return p
    raise FileNotFoundError(f"No file for {name} in {DATA}")


def write_run_log(table, rows, status, err=None):
    with engine.begin() as conn:
        conn.execute(text("INSERT INTO run_log (table_name, rows_loaded, status, error_message) "
                          "VALUES (:t, :r, :s, :e)"),
                     {"t": table, "r": rows, "s": status, "e": err})


def load_staging():
    for t in TABLES:
        try:
            total = 0
            for i, chunk in enumerate(pd.read_csv(find_file(t), sep=";", chunksize=100_000,
                                                  low_memory=False)):
                chunk.to_sql(t, engine, if_exists="replace" if i == 0 else "append",
                             index=False, chunksize=10_000)
                total += len(chunk)
            write_run_log(t, total, "SUCCESS")
            logger.info("Loaded %s: %d rows", t, total)
            print(f"{t}: {total} rows loaded")
        except Exception as e:
            write_run_log(t, 0, "FAIL", str(e)[:500])
            logger.error("Failed loading %s: %s", t, e)
            raise


def build_indexes():
    for tbl, col in INDEXES:
        try:
            with engine.begin() as conn:
                conn.execute(text(f"CREATE INDEX idx_{tbl}_{col} ON staging.{tbl} ({col})"))
        except Exception as e:
            logger.warning("Index idx_%s_%s skipped: %s", tbl, col, e)


def procedures_exist():
    with engine.connect() as conn:
        n = conn.execute(text("SELECT COUNT(*) FROM information_schema.routines "
                              "WHERE routine_schema = 'reporting' AND routine_name = 'sp_run_all'")).scalar()
    return n > 0


def refresh_reporting():
    with engine.begin() as conn:
        conn.execute(text("DROP TABLE IF EXISTS reporting.dim_district"))
        conn.execute(text("""
            CREATE TABLE reporting.dim_district AS
            SELECT A1 AS district_id, A2 AS district_name, A3 AS region, A4 AS population,
                   A9 AS num_cities, A10 AS urban_ratio_pct, A11 AS avg_salary,
                   A12 AS unemployment_rate_1995, A13 AS unemployment_rate_1996,
                   A14 AS entrepreneurs_per_1000, A15 AS crimes_1995, A16 AS crimes_1996
            FROM staging.district"""))
        conn.execute(text("CALL reporting.sp_run_all()"))


def main():
    logger.info("Pipeline started")
    load_staging()
    build_indexes()
    if not procedures_exist():
        print("Staging is loaded. Now run sql/01_setup.sql in MySQL Workbench, then run this script again.")
        logger.info("Staging loaded; reporting setup SQL not yet run")
        return
    refresh_reporting()
    logger.info("Reporting refresh and DQ checks complete")
    print("Pipeline finished successfully")
```

```
if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        logger.exception("Pipeline failed")
        print(f"Pipeline FAILED: {e}")
        sys.exit(1)
```

B.6 run_pipeline.bat

```
@echo off
cd /d C:\path\to\banking-etl-pipeline
python python\run_pipeline.py >> pipeline_run.log 2>&1
```